Contents

1 Some Data-Cleaning Tools **1**

1.1 Tidy Data 1

1.2 Tools for tidying data 2

1 Some Data-Cleaning Tools

```r
library(printr)
```

```
## Registered S3 method overwritten by 'printr':
##   method                from
##   knit_print.data.frame rmarkdown
```

- reading: @WickhamTidyData2014

1.1 Tidy Data

- conventions for organizing data for analysis
  1. Each variable forms a column.
  2. Each observation forms a row.
  3. Each type of observational unit forms a table.
- Example: student demographics and major, for students who took a philosophy course at UC Davis, 2005-2015

id	admit_term	rel_term	term	major
000[redacted]B68	201010	YR1FALL	201010	LPOL-Political Science
000[redacted]B68	201010	YR1WINTER	201101	LPOL-Political Science
000[redacted]B68	201010	YR1SPRING	201103	LPOL-Political Science
000[redacted]B68	201010	YR1SUMMER_I	201105	LPOL-Political Science
000[redacted]B68	201010	YR1SUMMER_II	201107	LPOL-Political Science
000[redacted]B68	201010	YR2FALL	201110	LPOL-Political Science
000[redacted]B68	201010	YR2WINTER	201201	LPOL-Political Science

id	gender	ethnicity_code	ethnicity	race	first_gen	low_income	admission_type	ever_phil	poc	race4	demographic	major_at_first_phil
8F[redacted]73	F	OT	Other	Other	FALSE	FALSE	Transfer	FALSE	FALSE	Other	F.Other	LENL-English
0F[redacted]99C	M	(Missing)	Other	Other	TRUE	FALSE	Transfer	FALSE	FALSE	Other	M.Other	APME-Pre-Managerial Economics
1C[redacted]9C2	F	WH	White/Caucasian	White	TRUE	FALSE	Transfer	FALSE	FALSE	White	F.White	LPSC-Psychology
3E[redacted]3EA	F	MX	Mexican-Am/Mexican/Chicano	Hispanic	TRUE	FALSE	Transfer	TRUE	TRUE	BHIP	F.BHIP	LPHI-Philosophy
95[redacted]693	F	KO	Korean-American/Korean	Asian	TRUE	TRUE	Transfer	FALSE	TRUE	Asian	F.Asian	AHDE-Human Development
A1[redacted]6B0	M	CH	Chinese-American/Chinese	Asian	FALSE	TRUE	Freshman	FALSE	TRUE	Asian	M.Asian	EMEC-Mechanical Engineering
6C[redacted]127	M	FP	Filipino/Filipino-American	Asian	FALSE	FALSE	Transfer	FALSE	TRUE	Asian	M.Asian	ABIS-Biological Sciences

- slightly rephrased
  - 1 table = 1 unit of observation
    - student-term (major)
    - student (demographics)
  - 1 row = 1 unique observed unit
    - 1 row = 1 student-term combination
    - 1 row = 1 student
  - Every column is either an *identifier* or a *measured variable*
    - rule of thumb: 1 row = 1 unique combination of identifiers
    - major table
      - identifiers: student ID, `term`

        - measured variables: admit term, relative term, major
        - NB the `term` identifier is constructed from two measured variables
    - demographics table
        - identifier: student ID
        - measured variables: gender, race and ethnicity variables, first gen status, low income status, admission type, whether they ever majored in philosophy, major the semester they took their first philosophy course
        - NB here "measured variable" includes both variables in the raw data as well as variables I constructed for analysis purposes (`poc`, `demographic`)

1.2 Tools for tidying data

```r
library(tidyverse)
```

```
## -- Attaching packages --------------------------------------- tidyverse 1.3.0 --
```

```
## v ggplot2 3.3.2     v purrr   0.3.4
## v tibble  3.0.4     v dplyr   1.0.2
## v tidyr   1.1.2     v stringr 1.4.0
## v readr   1.4.0     v forcats 0.5.0
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
```

```r
theme_set(theme_minimal())

data_folder = file.path('data')
```

1.2.1 3.1: The Pew Dataset

- `pivot_longer()`
- `if_else()`
- `fct_inorder()`

```r
pew_df = read_csv(file.path(data_folder, 'pew.csv'))
```

```
## Warning: Missing column names filled in: 'X1' [1]
```

```
##
## -- Column specification --------------------------------------------------------
## cols(
##   X1 = col_double(),
##   religion = col_character(),
##   `<$10k` = col_double(),
##   `$10-20k` = col_double(),
##   `$20-30k` = col_double(),
##   `$30-40k` = col_double(),
##   `$40-50k` = col_double(),
##   `$50-75k` = col_double(),
##   `$75-100k` = col_double(),
##   `$100-150k` = col_double(),
##   `>150k` = col_double(),
##   `Don't know/refused` = col_double()
## )
```

```r
head(pew_df)
```

X1	religion	<$10k	$10-20k	$20-30k	$30-40k	$40-50k	$50-75k	$75-100k	$100-150k	>150k
1	Agnostic	27	34	60	81	76	137	122	109	84
2	Atheist	12	27	37	52	35	70	73	59	74
3	Buddhist	27	21	30	34	33	58	62	39	53
4	Catholic	418	617	732	670	638	1116	949	792	633
5	Don't know/refused	15	14	15	11	10	35	21	17	18
6	Evangelical Prot	575	869	1064	982	881	1486	949	723	414

```r
pew_clean = pew_df %>%
    ## Drop the row number column
    select(-X1) %>%
    ## Lengthen: 1 row per religion x income level
    pivot_longer(cols = `<$10k`:`Don't know/refused`,
                 names_to = 'income',
                 values_to = 'freq') %>%
    ## Cleaning `income`:
    ## - Make "Don't know/refused" into NA
    ## - Coerce to ordered factor
    mutate(income = if_else(str_detect(income, "Don't know"),
                            NA_character_,
                            income),
           income = fct_inorder(income))
head(pew_clean)
```

religion	income	freq
Agnostic	<$10k	27
Agnostic	$10-20k	34
Agnostic	$20-30k	60
Agnostic	$30-40k	81
Agnostic	$40-50k	76
Agnostic	$50-75k	137

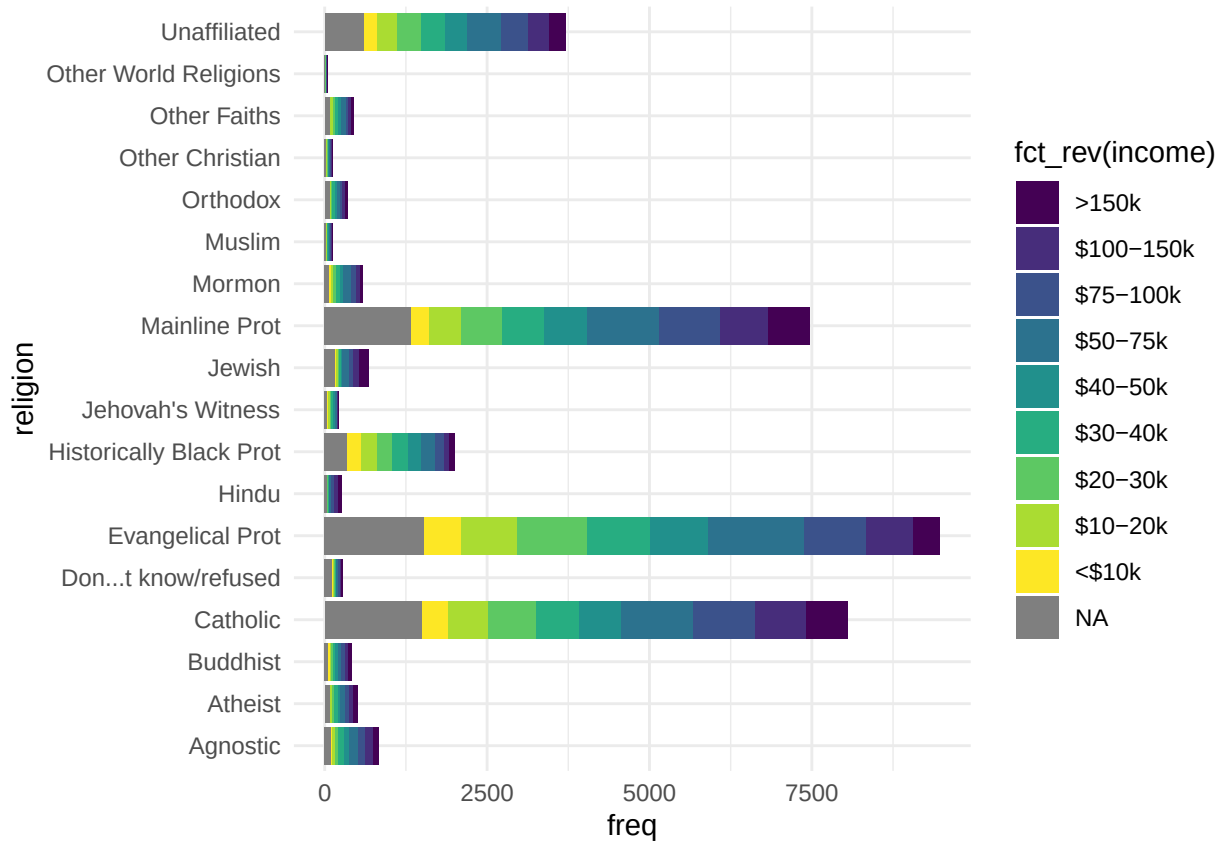
```r
ggplot(pew_clean, aes(religion, freq, fill = fct_rev(income))) +
    geom_col() +
    scale_fill_viridis_d(na.value = 'grey50') +
    coord_flip()
```

```
## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don't know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don't know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don't know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don't know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don't know/refused' in 'mbcsToSbcs': dot substituted for
```

```
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>
```

```
## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
```

```
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>

## Warning in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <e2>

## Warning in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <80>

## Warning in grid.Call.graphics(C_text, as.graphicsAnnot(x$label), x$x, x$y, :
## conversion failure on 'Don’t know/refused' in 'mbcsToSbcs': dot substituted for
## <99>
```



1.2.2 3.1: The Billboard Dataset

- A little helper function for parsing
- `str_remove()`
- Coercion
- Date arithmetic

```
billboard_df = read_csv(file.path(data_folder, 'billboard.csv'))
```

```
## Warning: Missing column names filled in: 'X1' [1]
```

```
##
## -- Column specification --------------------------------------------------------
## cols(
##   .default = col_double(),
##   artist = col_character(),
##   track = col_character(),
##   time = col_time(format = ""),
##   date.entered = col_date(format = ""),
##   wk66 = col_logical(),
##   wk67 = col_logical(),
##   wk68 = col_logical(),
##   wk69 = col_logical(),
##   wk70 = col_logical(),
##   wk71 = col_logical(),
##   wk72 = col_logical(),
##   wk73 = col_logical(),
```

```
##   wk74 = col_logical(),
##   wk75 = col_logical(),
##   wk76 = col_logical()
## )
## i Use `spec()` for the full column specifications.
```

```r
head(billboard_df)
```

X1	year	artist	track	time	date.entered	wk1	wk2	wk3	wk4	wk5	wk6
1	2000	2 Pac	Baby Don't Cry	04:22:00	2000-02-26	87	82	72	77	87	94
2	2000	2Ge+her	The Hardest Part Of ...	03:15:00	2000-09-02	91	87	92	NA	NA	NA
3	2000	3 Doors Down	Kryptonite	03:53:00	2000-04-08	81	70	68	67	66	57
4	2000	3 Doors Down	Loser	04:24:00	2000-10-21	76	76	72	69	67	65
5	2000	504 Boyz	Wobble Wobble	03:35:00	2000-04-15	57	34	25	17	17	31
6	2000	98^0	Give Me Just One Nig...	03:24:00	2000-08-19	51	39	34	26	26	19

```r
#' Helper function to parse weeks to integers in the Billboard dataset
#' @param wk_chr Vector of week characters, with the pattern `c('wk1', 'wk2')`
#' @return Vector of integers, eg, `c(1L, 2L)`
parse_week = function(wk_chr) {
    wk_int = wk_chr %>%
        str_remove('wk') %>%
        as.integer()
    return(wk_int)
}

billboard_clean = billboard_df %>%
    ## Drop the row number column
    select(-X1) %>%
    ## Pivot week down and remove NA rows
    pivot_longer(cols = starts_with('wk'),
                 names_to = 'week',
                 values_to = 'rank') %>%
    filter(!is.na(rank)) %>%
    ## Parse week and calculate date
    ## NB Base R can handle date arithmetic
    ## For month arithmetic see ?lubridate::`%m+%`
    mutate(week = parse_week(week),
           date = date.entered + (week-1)*7) %>%
    ## Clean columns
    select(year, artist, time, track, date, week, rank)
head(billboard_clean)
```

year	artist	time	track	date	week	rank
2000	2 Pac	04:22:00	Baby Don't Cry	2000-02-26	1	87
2000	2 Pac	04:22:00	Baby Don't Cry	2000-03-04	2	82
2000	2 Pac	04:22:00	Baby Don't Cry	2000-03-11	3	72
2000	2 Pac	04:22:00	Baby Don't Cry	2000-03-18	4	77
2000	2 Pac	04:22:00	Baby Don't Cry	2000-03-25	5	87
2000	2 Pac	04:22:00	Baby Don't Cry	2000-04-01	6	94

NB. Base R includes methods for doing date arithmetic, eg,

```r
as.Date('2020-04-30') + 1
```

```
## [1] "2020-05-01"
```

If you need month arithmetic, check out the `lubridate` package, and specifically `%m+%` and related operators.

```
count(billboard_clean, artist, track) %>%
    arrange(desc(n)) %>%
    head()
```

artist	track	n
Creed	Higher	57
Lonestar	Amazed	55
3 Doors Down	Kryptonite	53
Hill, Faith	Breathe	53
Creed	With Arms Wide Open	47
Joe	I Wanna Know	44

1.2.3 3.2: The TB Dataset

- `haven` package
- `separate()`
- `case_when()`

```
tb_df = haven::read_sav(file.path(data_folder, 'tb.sav'))
head(tb_df)
```

country	year	m014	m1524	m2534	m3544	m4554	m5564	m65	mu	f014	f1524	f2534	f3544	f4554
AD	2000	0	0	1	0	0	0	0	NA	NA	NA	NA	NA	NA
AE	2000	2	4	4	6	5	12	10	NA	3	16	1	3	0
AF	2000	52	228	183	149	129	94	80	NA	93	414	565	339	205
AG	2000	0	0	0	0	0	0	1	NA	1	1	1	0	0
AL	2000	2	19	21	14	24	19	16	NA	3	11	10	8	8
AM	2000	2	152	130	131	63	26	21	NA	1	24	27	24	8

```
tb_clean = tb_df %>%
    ## One row per observation = country x gender x age group
    pivot_longer(m014:fu,
                 names_to = 'column',
                 values_to = 'cases') %>%
    ## Separate gender and age
    separate(column, into = c('gender', 'age'),
             sep = 1) %>%
    ## Wrangle age:
    ## - Split into upper and lower bounds
    ## - Recombine with -
    ## - Fix missing age and 65+
    ## - Set order
    ## - Clean up the intermediate columns for upper and lower bounds
    separate(age, into = c('age_lower', 'age_upper'),
             sep = -2) %>%
    mutate(age = str_c(age_lower, '-', age_upper),
           age = case_when(age == '-u'  ~ NA_character_,
                           age == '-65' ~ '65+',
                           TRUE         ~ age),
           age = fct_inorder(age)) %>%
    select(-age_lower, -age_upper) %>%
    ## Put in order, with ID columns first and then the variable
    select(country, year, gender, age, cases)
head(tb_clean)
```

country	year	gender	age	cases
AD	2000	m	0-14	0
AD	2000	m	15-24	0
AD	2000	m	25-34	1
AD	2000	m	35-44	0
AD	2000	m	45-54	0
AD	2000	m	55-64	0

1.2.4 3.3: The Weather Dataset

- `pivot_wider()`
- `str_remove()`

```
weather_df = read_csv(file.path(data_folder, 'weather.csv'))
```

```
## Warning: Missing column names filled in: 'X1' [1]
```

```
##
## -- Column specification --------------------------------------------------------
## cols(
##   .default = col_double(),
##   id = col_character(),
##   element = col_character(),
##   d9 = col_logical(),
##   d12 = col_logical(),
##   d18 = col_logical(),
##   d19 = col_logical(),
##   d20 = col_logical(),
##   d21 = col_logical(),
##   d22 = col_logical(),
##   d24 = col_logical()
## )
## i Use `spec()` for the full column specifications.
```

```
head(weather_df)
```

X1	id	year	month	element	d1	d2	d3	d4	d5	d6	d7	d8	d9	d10	d11	d12
1	MX17004	2010	1	tmax	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
2	MX17004	2010	1	tmin	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
3	MX17004	2010	2	tmax	NA	27.3	24.1	NA	NA	NA	NA	NA	NA	NA	29.7	NA
4	MX17004	2010	2	tmin	NA	14.4	14.4	NA	NA	NA	NA	NA	NA	NA	13.4	NA
5	MX17004	2010	3	tmax	NA	NA	NA	NA	32.1	NA	NA	NA	NA	34.5	NA	NA
6	MX17004	2010	3	tmin	NA	NA	NA	NA	14.2	NA	NA	NA	NA	16.8	NA	NA

```
weather_clean = weather_df %>%
    select(-X1) %>%
    ## Lengthen days
    pivot_longer(d1:d31,
                 names_to = 'day',
                 values_to = 'obs') %>%
    ## Widen `element`
    pivot_wider(names_from = element,
                values_from = obs) %>%
    ## Fix the class of `day`
    mutate(day = str_remove(day, 'd'),
           day = as.integer(day)) %>%
    ## Check against the paper by filtering to non-missing rows
```

```r
    filter(!is.na(tmax) | !is.na(tmin))
head(weather_clean)
```

id	year	month	day	tmax	tmin
MX17004	2010	1	30	27.8	14.5
MX17004	2010	2	2	27.3	14.4
MX17004	2010	2	3	24.1	14.4
MX17004	2010	2	11	29.7	13.4
MX17004	2010	2	23	29.9	10.7
MX17004	2010	3	5	32.1	14.2